

Résumé du Chapitre V : Régression Logistique avec PyTorch

Ce document, qui fait partie du cours de Traitement Automatique du Langage Naturel (TALN) à l'Université des Sciences et de la Technologie Houari Boumediene (USTHB), présente une introduction pratique à la Régression Logistique implémentée avec la bibliothèque PyTorch.

PyTorch

PyTorch est une bibliothèque d'apprentissage profond développée par Meta (Facebook AI Research). Elle est utilisée pour :

- 1. **Calcul numérique** : Manipuler des Tensors (tableaux multidimensionnels optimisés)
- 2. **Apprentissage automatique** : Concevoir, entraîner et déployer des réseaux de neurones profonds (CNN, LSTM, Transformers)
- 3. **Exécution des calculs** sur GPU ou CPU via CUDA (NVIDIA) ou MPS (Apple)

Composants Clés de PyTorch

Composant	Description
<code>torch.Tensor</code>	Structure de base pour stocker et manipuler des données numériques (compatible GPU)
<code>autograd</code>	Système de dérivation automatique qui calcule les gradients nécessaires à la descente de gradient
<code>torch.nn</code>	Module pour construire des architectures de réseaux neuronaux (couches linéaires, convolutions, RNN, etc.). Tous les modèles PyTorch doivent hériter de la classe <code>nn.Module</code>
<code>torch.optim</code>	Implémente les algorithmes d'optimisation (SGD, Adam, RMSProp, etc.) pour mettre à jour les poids du modèle
<code>torch.utils.data</code>	Fournit des outils pour gérer les ensembles de données
<code>torchvision / torchaudio / torchtext</code>	Extensions spécialisées pour le traitement d'images, d'audio, ou de texte

Optimisation GPU/CPU

- **CUDA** (Compute Unified Device Architecture) : Technologie NVIDIA permettant d'utiliser les GPU NVIDIA pour exécuter des calculs parallèles intensifs et accélérer l'entraînement
- **MPS** (Metal Performance Shaders) : Technologie Apple permettant d'utiliser le GPU intégré des Mac (M1, M2, M3, etc.) pour effectuer des calculs intensifs

Préparation des Données et Outils

Prérequis

Les installations nécessaires incluent Python (≥ 3.8), PyTorch (≥ 1.12 pour MPS), NumPy, et Scikit-learn.

torch.utils.data

- **TensorDataset** : Une classe qui regroupe plusieurs Tensors (données x et étiquettes y) en un seul objet dataset structuré
- **DataLoader** : Un outil essentiel pour gérer le chargement des données par mini-lots (batches) pendant l'entraînement. Il permet d'itérer facilement sur les données et de mélanger les exemples. Les mini-lots sont utilisés pour éviter le chargement lent du dataset entier en mémoire.

sklearn.datasets

- **make_classification** : Une fonction de Scikit-learn utilisée pour générer un jeu de données de classification supervisée synthétique.
 - Le paramètre `n_clusters_per_class=1` génère des classes monomodales, rendant la classification plus simple et potentiellement linéairement séparable, nécessitant des modèles linéaires comme la régression logistique.
 - Le paramètre `random_state` assure la reproductibilité du dataset généré.
- **train_test_split** : Divise le jeu de données en un ensemble d'entraînement et un ensemble de test. `test_size=0.2` utilise 80% pour l'entraînement et 20% pour le test.

Préparation des Tensors

- Les données NumPy générées par `make_classification` sont converties en Tensors PyTorch.
- Les étiquettes y sont transformées en colonne de forme $(N, 1)$ à l'aide de `reshape(-1, 1)` pour être compatibles avec PyTorch.
- Le type de données `float32` est utilisé pour optimiser les calculs sur GPU (il est plus rapide que `float64`).



Définition et Entraînement du Modèle

Modèle de Régression Logistique

- Le modèle est défini comme une classe `LogisticRegression` qui hérite de `nn.Module`.
- La couche linéaire est créée via `self.linear = nn.Linear(input_dim, 1)`. Le `1` indique une seule sortie (un *logit*) pour la classification binaire.
- La méthode `forward(self, x)` calcule le logit $z = xW + b$ et renvoie `self.linear(x)`. La fonction Sigmoid n'est pas appliquée dans `forward()` car elle est incluse dans la fonction de perte.

Fonction de Perte et Optimiseur

- **Fonction de Perte** : `criterion = nn.BCEWithLogitsLoss()`. Cette fonction combine automatiquement l'application du Sigmoid et le calcul de la Binary Cross Entropy (BCE).
- **Optimiseur** : `optimizer = optim.SGD(model.parameters(), lr=0.1, momentum=0.9)`. L'optimiseur utilise la Descente de Gradient Stochastique (SGD) avec un momentum pour accélérer le gradient et éviter les oscillations.

Phase d'Entraînement (Epoch)

1. Effacer les gradients précédents : `optimizer.zero_grad()`
2. Calculer les logits (passe avant) : `logits = model(x_batch)`
3. Calculer la perte : `loss = criterion(logits, y_batch)`
4. Calculer les gradients (rétropropagation) : `loss.backward()`
5. Mettre à jour les poids : `optimizer.step()`
6. Une fois que tous les mini-lots ont été traités, une **époque** (epoch) est terminée.

Évaluation (evaluate function)

1. Le modèle est mis en mode évaluation : `model.eval()`
2. Le calcul du gradient est désactivé avec `with torch.no_grad()`: pour économiser de la mémoire et du temps pendant l'inférence
3. Les logits sont transformés en probabilités avec `probs = torch.sigmoid(logits)`
4. Les prédictions binaires sont obtenues par seuillage à 0.5 : `preds = (probs >= 0.5).float()`
5. L'**Accuracy** est calculée en comparant les prédictions et les étiquettes : `correct += (preds == y_batch).sum().item()`